Advanced Algorithms

Lecture Notes

Luca Simonetti

2025–2026

Contents

1 String Matching **5**
1.1 Introduction & Exact Pattern Matching 5
1.1.1 Definition 5
1.1.2 Naive approach 6
1.1.3 Knuth-Morris-Pratt (KMP) algorithm 8

2 Randomized Algorithms **13**

3 Data Compression **15**

Chapter 1

String Matching

1.1 Introduction & Exact Pattern Matching

1.1.1 Definition

The exact pattern matching problem is defined as follows:

Problem 1.1.1 (Exact pattern matching)**.** Given an alphabet Σ, and two strings T, $P \in \Sigma^*$, for which $|T| = n$ and $|P| = m$, find all occurrences of P in T, i.e. compute $\{\, i \mid T[i,\, i+m-1] = P \,\}$.

Notation

Throughout these notes, Σ denotes a finite alphabet of constant size ($|\Sigma| \in \mathcal{O}(1)$). For a string S of length $|S|$:

- $S[i]$ denotes the i-th character of S (1-based indexing).
- $S[i, j]$ denotes the substring $S[i]\, S[i+1] \cdots S[j]$ (empty string if $i > j$).
- $S[,j] = S[1, j]$ is the prefix of length j; $S[i,] = S[i, |S|]$ is the suffix starting at i.

A *proper* prefix (resp. suffix) of S is any prefix (resp. suffix) strictly shorter than S itself.

Example 1.1.2. Let $\Sigma = \{a, b\}$, $T =$ `ababbab` and $P =$ `abb`. The only occurrence of P in T is at position 3.

1.1.2 Naive approach

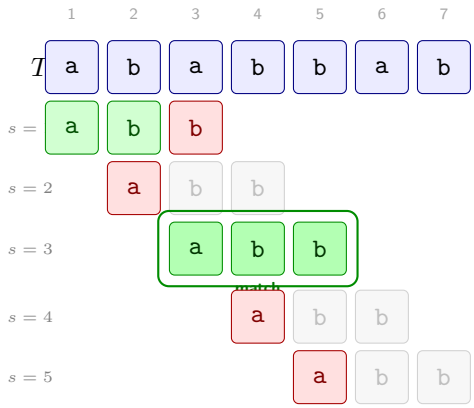


Figure 1.1: Naive exact pattern matching of $P =$ `abb` in $T =$ `ababbab` (1-indexed). Each row corresponds to one shift s; the pattern P is aligned starting at $T[s]$. **Green** = character matched, **red** = first mismatch (algorithm skips to next shift), gray = not compared.

What the Figure 1.1 illustrates is one basic naive approach. In this approach we nest two loops: the outer loop iterates over all possible "first positions" s in T where the pattern P could potentially match, and the inner loop checks character by character if P matches T starting from position s. For example the first row of the figure corresponds to starting the search from index $s = 1$ of T. Given s we then loop one by one each character of P, moving along T as well, and check if all the characters match. In this case we find a mismatch at the third character of P (which is `b`, whereas the corresponding character in T is `a`). When this happens, we break this inner loop and move to the next shift $s = 2$. If the length of the pattern was m and the length of the text was n, the outer loop could have up to $\mathcal{O}(n)$ iterations, and each inner iteration could take up to $\mathcal{O}(m)$ time, leading to a worst-case time complexity of $\mathcal{O}(nm)$ for this naive approach.

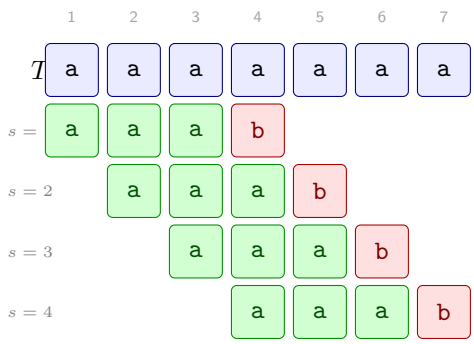


Figure 1.2: Worst-case input for the naive algorithm: $T =$ `aaaaaaa` ($n = 7$) and $P =$ `aaab` ($m = 4$). Every shift s performs $m - 1$ successful comparisons before the inevitable mismatch on $P[m] =$ `b`, giving $(n - m + 1)(m - 1) = \Theta(nm)$ total comparisons.

In figure 1.2 we see an example of a worst-case input for the naive algorithm. In this case the text T consists of n `a` characters, and the pattern P consists of $m - 1$ `a` characters followed by a `b`. There are $n - m + 1$ possible shifts s where

the pattern could potentially match, and for each of these shifts the algorithm will successfully match the first $m-1$ characters of P. In the example of the figure we have $n=7$ and $m=4$, so there are $7-4+1=4$ shifts to check, and for each shift we make m checks, of which $m-1=3$ are successful matches before we encounter the mismatch at the last character of P. This results in a total of $(n-m+1)(m-1)$ comparisons, which is $\Theta(nm)$ in the worst case.

Algorithm 1: Naive exact pattern matching

```
Input: Text T[1...n], Pattern P[1...m]
Output: All positions s such that T[s, s+m-1] = P
for s ← 1 to n − m + 1 do
    j ← 1
    while j ≤ m and T[s + j − 1] = P[j] do
        j ← j + 1
    if j > m then
        output s
```

Correctness

To pin down what "the inner loop does" precisely, let's give a name to the obvious quantity: for any position i in T, write

$$\text{common}(i) \;=\; \max\{\, k \geq 0 \mid T[i, i+k-1] = P[1,k] \,\}$$

for the length of the longest common prefix of $T[i,]$ and P — basically, how many characters of P would match if you aligned it at position i.

Lemma 1.1.3. *At shift s, the inner loop exits with $j = \text{common}(s) + 1$.*

Proof. Track the invariant: entering each iteration, we have $T[s, s+j-2] = P[1, j-1]$ (the first $j-1$ characters already matched). The loop keeps going as long as the next character also matches; it stops the moment it doesn't, or when j runs past m. Since $\text{common}(s)$ is exactly how many consecutive characters agree, the loop stops right at $j = \text{common}(s) + 1$. □

Theorem 1.1.4. *The naive algorithm finds every occurrence of P in T, and nothing more.*

Proof. The outer loop visits every shift $s \in \{1, \ldots, n-m+1\}$, so no candidate starting position is ever skipped. By Lemma 1.1.3, after the inner loop we have $j = \text{common}(s) + 1$; the algorithm outputs s exactly when $j > m$, i.e. $\text{common}(s) \geq m$, i.e. $T[s, s+m-1] = P$ — which is precisely the definition of an occurrence. □

Lower bound

Theorem 1.1.5. *Any algorithm for exact pattern matching must make $\Omega(n+m)$ character comparisons in the worst case.*

Proof. Two independent arguments, one for each term.

- **You have to read all of P** ($\Omega(m)$). Suppose the algorithm skips position $P[q]$. An adversary flips just that character, producing a different pattern P'. The algorithm sees the same comparisons on both inputs and therefore gives the same output — which can't be correct for both P and P'.

- **You have to read all of** T ($\Omega(n)$). Suppose position $T[i]$ is never read. The adversary flips that character, potentially creating or destroying a match right around position i. Since the algorithm never looked at $T[i]$, it has no way to tell the difference.

The two bounds add up: $\Omega(m) + \Omega(n) = \Omega(n + m)$. □

This $\Omega(n+m)$ lower bound applies to any *algorithm, not just naive.*

The naive algorithm achieves $\mathcal{O}(nm)$ in the worst case, which is far from this lower bound. KMP will close the gap, achieving $\Theta(n + m)$.

Properties

We evaluate string matching algorithms along four axes:

Online. The algorithm processes T strictly left-to-right, one character at a time, and can emit results before reading future characters of T. Useful when T arrives as a stream.

Real-time. A stronger requirement: each new character of T is fully processed in $\mathcal{O}(1)$ *worst-case* time before the next one is read.

Blind (no backtracking). The algorithm never re-reads a position of T: each character of T is accessed at most once. A blind algorithm is necessarily online; the converse is false (the naive algorithm is online but not blind).

Succinct. The algorithm uses only $\mathcal{O}(1)$ extra space beyond storing P and T (no large auxiliary data structures).

Algorithm	Online	Real-time	Blind	Succinct
Naive	✓	×	×	✓
KMP	✓	×	✓	×
KMP (sp')	✓	✓	✓	×

The naive algorithm backtracks in T: at each new shift s it re-reads characters already compared at shift $s - 1$.

The naive algorithm is **online** (it reads T left-to-right) but not **blind**: at each new shift it backtracks and re-compares characters of T that were already seen. It is also not **real-time**: a single character of T can trigger up to $\mathcal{O}(m)$ comparisons.

This naive approach basically starts over from scratch at each shift, without leveraging precious information from previous comparisons.

1.1.3 Knuth-Morris-Pratt (KMP) algorithm

Intuition

What if we could somehow "remember" the information from a previous comparison, and use it to skip some of the checks at the next shift? Let's introduce the intuition behind the Knuth-Morris-Pratt (KMP) algorithm, with an example.

Example 1.1.6. Let $P =$ `abcdeabdef` and suppose we are running the naive algorithm at shift $s = 3$ over some text T whose characters at positions 3–9 are `abcdeab`. We match $P[1 \ldots 7]$ successfully, then hit a mismatch at $P[8] =$ `d` vs $T[10] =$ `x`.

As shown in Figure 1.3, at this point the naive algorithm would discard everything and try shift $s = 4$. But notice that the matched portion `abcdeab` starts *and* ends with $\alpha =$ `ab`: the prefix $P[1 \ldots 2]$ and the suffix $P[6 \ldots 7]$ of the matched region are identical. This means that when we slide P forward, the α that was at the end of the match is now perfectly aligned with the α at the start of P — we can jump directly to shift $s = 8$ and resume comparing from $P[3]$, skipping five shifts entirely.

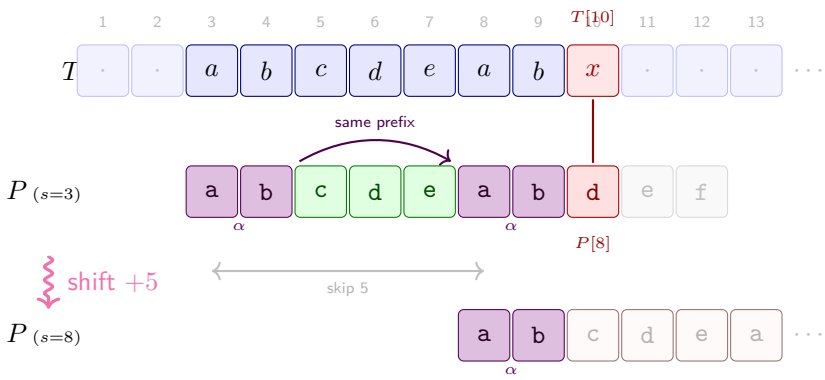
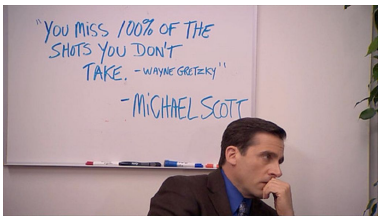


Figure 1.3: KMP shift intuition with $P =$ `abcdeabdef`. At shift $s = 3$, the algorithm matches $P[1 \ldots 7] =$ `abcdeab` then hits a mismatch at $P[8]$. The matched portion ends with $\alpha =$ `ab`, which also appears at the very start of P — so instead of shifting by 1, we jump directly to $s = 8$, re-aligning α and resuming from $P[3]$.

Basically what the example shows is that if we take the longest prefix of P that is also a suffix of the **matched** portion of P, we can shift the pattern so that this prefix is now aligned with the suffix — and we can resume the search from the character right after this prefix, since we know it must match (otherwise we would have had a mismatch earlier). In other words we have saved some work (namely, five shift indexes) by leveraging the information from the previous comparisons, instead of starting over from scratch, from the very next shift.

*When we say suffix here we mean suffix of the **matched** portion of P, not of P itself. In the example, α is a suffix of $P[1 \ldots 7]$ but not of the whole P.*

But is this correct? Or do we miss some potential matches by jumping directly to $s = 8$?



"You miss 100% of the shots you don't take." — Michael Scott

Well, it turns out that this is indeed correct, and we won't miss any matches by doing this. The KMP algorithm is based on this very idea, and it guarantees that we will find all occurrences of P in T without missing any, while also achieving a much better time complexity than the naive approach.

Preprocessing: computing *sp* values

A proper *prefix/suffix excludes the string itself.*

To exploit this idea, we need to precompute for every prefix $P[1 \dots i]$ its *longest proper prefix that is also a suffix*. We write this length as $\mathrm{sp}_i(P)$.

Definition 1.1.7 (*sp* values)**.** Let P be a pattern of length m. For each $i \in \{1, \dots, m\}$, define

$$\mathrm{sp}_i(P) \;=\; \max\{\, k < i \mid P[1 \dots k] = P[i-k+1 \dots i] \,\},$$

i.e. the length of the longest proper prefix of $P[1 \dots i]$ that is also a suffix of $P[1 \dots i]$.

The key observation is that $\mathrm{sp}_i(P)$ can be computed recursively from $\mathrm{sp}_{i-1}(P)$, giving an efficient $\mathcal{O}(m)$ preprocessing algorithm. There are two cases:

1. **Extension is possible.** Let $k = \mathrm{sp}_{i-1}(P)$. If $P[k+1] = P[i]$, then the longest prefix-suffix of $P[1 \dots i]$ is simply one character longer than that of $P[1 \dots i-1]$, so $\mathrm{sp}_i(P) = k+1$.

   Example: with $P =$ `abcdeabdef` at $i = 7$, we have $\mathrm{sp}_6(P) = 1$ (the prefix-suffix of `abcdea` is `a`, length 1). We check $P[2] =$ `b` against $P[7] =$ `b`: they match, so $\mathrm{sp}_7(P) = 2$.

2. **Extension is not possible.** If $P[k+1] \neq P[i]$, we cannot extend the current prefix-suffix. We then ask: what is the longest prefix-suffix of the prefix-suffix itself? In other words, we fall back to $k \leftarrow \mathrm{sp}_k(P)$ and try to extend that instead. We repeat this process until either a match is found or we reach $k = 0$, in which case $\mathrm{sp}_i(P) = 0$.

   Example: still with $P =$ `abcdeabdef` at $i = 8$, we have $\mathrm{sp}_7(P) = 2$ (the prefix-suffix of `abcdeab` is `ab`). We check $P[3] =$ `c` against $P[8] =$ `d`: no match. We fall back to $k \leftarrow \mathrm{sp}_2(P) = 0$, so we check $P[1] =$ `a` against $P[8] =$ `d`, still no match. We have reached $k = 0$, so $\mathrm{sp}_8(P) = 0$.

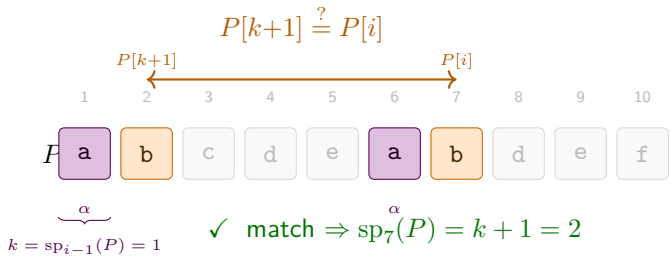


Figure 1.4: Computing $\mathrm{sp}_7(P)$ for $P =$ `abcdeabdef`. We know $k = \mathrm{sp}_6(P) = 1$: the suffix of $P[1 \dots 6]$ matching a prefix of P has length 1 (namely $\alpha =$ `a`, violet). We compare $P[k+1] = P[2] =$ `b` against $P[i] = P[7] =$ `b` (orange): they match, so $\mathrm{sp}_7(P) = 2$.

The resulting algorithm is the following:

Algorithm 2: Computing sp values

```
Input: Pattern P[1...m]
Output: Array sp[1...m] where sp[i] = sp_i(P)
sp[1] ← 0
k ← 0            // length of current candidate prefix-suffix
for i ← 2 to m do
    while k > 0 and P[k+1] ≠ P[i] do
        k ← sp[k]                                  // fall back
    if P[k+1] = P[i] then
        k ← k+1
    sp[i] ← k
return sp
```

Remark 1.1.8. The "fall back to the sp value of the current sp value" is the recursive heart of KMP: the same shift logic used during the search phase is also used during preprocessing.

Exercise 1.1.9. Design an algorithm that, given a pattern P of length m, computes $\mathrm{sp}_i(P)$ for every $i \in \{1, \ldots, m\}$ in $\mathcal{O}(m)$ time.

Hints:

1. Try to express $\mathrm{sp}_i(P)$ in terms of $\mathrm{sp}_{i-1}(P)$. What is the relationship between the longest prefix-suffix of $P[1 \ldots i]$ and that of $P[1 \ldots i-1]$?
2. There are two cases depending on whether $P[\mathrm{sp}_{i-1}(P)+1]$ equals $P[i]$ or not. What can you do in each case?
3. For the harder case, think recursively: if you cannot extend a prefix-suffix of length k, which shorter candidate should you try next?

Exercise 1.1.10 (Period of a string). The *period* of a string S of length n is the smallest integer $p \geq 1$ such that $S[i] = S[i+p]$ for all $1 \leq i \leq n-p$. Equivalently, p is the length of the shortest string Π such that S is a prefix of $\Pi^\infty = \Pi\Pi\Pi\Pi\cdots$

Design an $\mathcal{O}(n)$-time algorithm that, given S, computes its period.

Hints:

1. Compute the sp values of S.
2. What is the relationship between $sp_n(S)$ and the period of S? In particular, is $n - sp_n(S)$ always a period?
3. Is it always the *shortest* period? What additional condition on $sp_n(S)$ guarantees minimality?

Chapter 2

Randomized Algorithms

Chapter 3

Data Compression